

TAREA 2

Enrutamiento y Balanceo de Carga con NGINX

Portal de noticias con IA, Docker Compose y balanceo de carga

TechPulse

Estudiante: Erick Noe Gómez López

Carnet: 201700866

Curso: Software Avanzado

Fecha: 14/08/2026

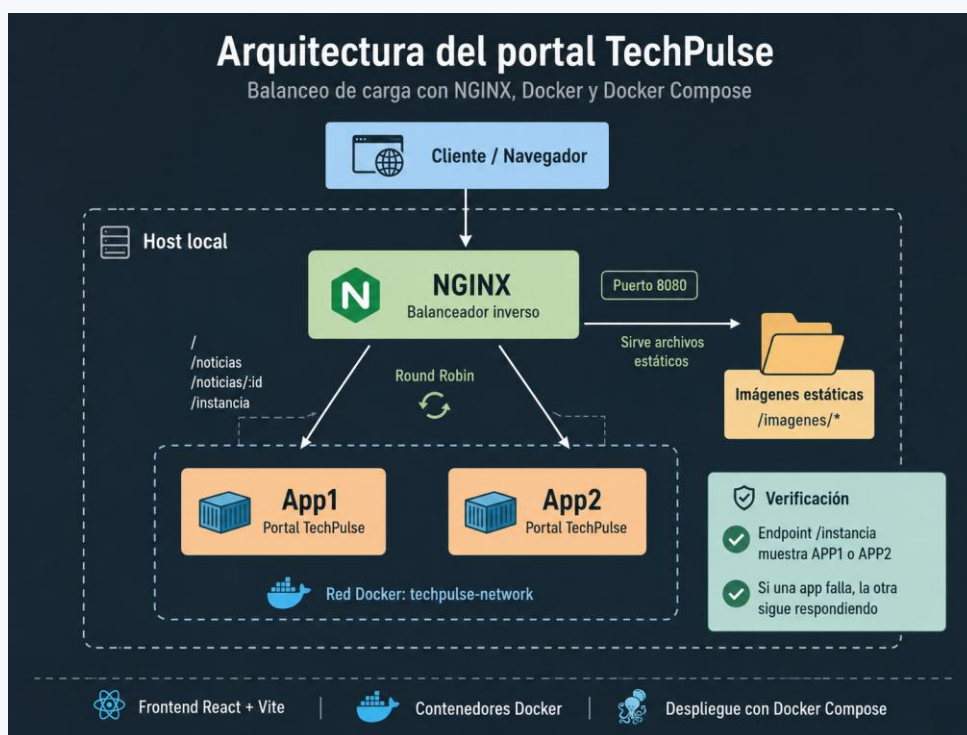
Documento de evidencias

Resumen del proyecto

TechPulse es un portal académico de noticias tecnológicas desarrollado con React y contenerizado mediante Docker. La solución utiliza dos instancias de la aplicación y un contenedor NGINX externo como único punto de entrada, encargado del enrutamiento y del balanceo de carga mediante Round Robin.

El contenido textual y las imágenes de las noticias fueron generados o asistidos mediante ChatGPT. El portal identifica el uso de inteligencia artificial y aclara que el contenido se presenta exclusivamente con fines académicos y no constituye información periodística verificada.

Arquitectura general



Orden de las evidencias

1. Ejecución de todos los contenedores.
2. Acceso al portal mediante NGINX.
3. Funcionamiento de las rutas configuradas.
4. Respuestas atendidas por dos instancias diferentes.
5. Prueba de balanceo con varias solicitudes consecutivas.
6. Comportamiento al detener una instancia.
7. Identificación de la herramienta de IA en las noticias.
8. Archivo docker-compose.yml.
9. Configuración de NGINX.
10. Dockerfile de la aplicación.
11. Evidencia complementaria de diseño responsive.

EVIDENCIA 1

Ejecución de todos los contenedores

Objetivo: Demostrar que la solución completa se inicia con Docker Compose y que los tres servicios se encuentran activos.

Procedimiento para obtener la captura:

- Desde la raíz del repositorio, detener cualquier ejecución previa y levantar la solución limpia.

```
docker compose down
docker compose up -d --build
docker compose ps
```

- Tomar la captura cuando aparezcan techpulse-app1, techpulse-app2 y techpulse-nginx en estado Up. Debe verse que únicamente NGINX publica el puerto 8080.

```
PS C:\Users\Noe\Desktop\SA-Tarea2-NGINX-Portal-Noticias> docker compose down
[+] down 4/4
✓ Container techpulse-nginx Removed 0.9s
✓ Container techpulse-app1 Removed 0.8s
✓ Container techpulse-app2 Removed 0.8s
✓ Network sa-tarea2-nginx-portal-noticias_techpulse-network Removed 0.3s
PS C:\Users\Noe\Desktop\SA-Tarea2-NGINX-Portal-Noticias> docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.14kB 0.0s done
#1 DONE 0.0s

[+] up 0/0
✓ Image sa-tarea2-nginx-portal-noticias-app1 Built 5.8s
✓ Image sa-tarea2-nginx-portal-noticias-app2 Built 5.8s
✓ Network sa-tarea2-nginx-portal-noticias_techpulse-network Created 0.2s
✓ Container techpulse-app1 Started 1.1s
✓ Container techpulse-app2 Started 1.1s
✓ Container techpulse-nginx Started 1.5s
PS C:\Users\Noe\Desktop\SA-Tarea2-NGINX-Portal-Noticias> docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
techpulse-app1	sa-tarea2-nginx-portal-noticias-app1	"/docker-entrypoint..."	app1	7 seconds ago	Up 6 seconds	80/tcp
techpulse-app2	sa-tarea2-nginx-portal-noticias-app2	"/docker-entrypoint..."	app2	7 seconds ago	Up 6 seconds	80/tcp
techpulse-nginx	nginx:alpine	"/docker-entrypoint..."	nginx	7 seconds ago	Up 5 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp

```
PS C:\Users\Noe\Desktop\SA-Tarea2-NGINX-Portal-Noticias>
```

La captura demuestra que Docker Compose levantó correctamente las dos instancias de la aplicación y el contenedor NGINX. Las instancias app1 y app2 solo exponen el puerto interno 80/tcp, mientras que NGINX es el único servicio publicado en el puerto 8080 del equipo anfitrión.

EVIDENCIA 2

Acceso al portal mediante NGINX

Objetivo: Comprobar que el usuario accede al portal únicamente a través del puerto publicado por NGINX.

Procedimiento para obtener la captura:

- Abrir el navegador en la dirección indicada.

```
http://localhost:8080
```

- Tomar una captura completa de la página principal donde se observe la URL y el portal TechPulse funcionando.



El portal se encuentra disponible mediante `http://localhost:8080`, que corresponde al puerto publicado por el contenedor NGINX. Las instancias internas no están expuestas directamente al usuario.

EVIDENCIA 3

Funcionamiento de las rutas configuradas

Objetivo: Demostrar que NGINX enruta correctamente las solicitudes hacia la aplicación y también sirve una ruta estática específica.

Procedimiento para obtener la captura:

- Abrir la ruta del listado de noticias y tomar una captura.

```
http://localhost:8080/noticias
```

- Abrir la ruta estática servida directamente por NGINX y tomar otra captura.

```
http://localhost:8080/imagenes/robotica.webp
```



La ruta /noticias dirige la solicitud hacia las instancias balanceadas del portal, mientras que /imagenes/ se atiende directamente desde el NGINX externo. Esto evidencia la existencia de reglas de enrutamiento distintas dentro de la configuración.

EVIDENCIA 4

Respuestas atendidas por dos instancias diferentes

Objetivo: Mostrar de forma visible que una solicitud de verificación puede ser atendida por APP1 o APP2.

Procedimiento para obtener la captura:

- Abrir el portal, desplazarse al pie de página y observar el indicador de instancia.
- Actualizar la página con F5 hasta obtener APP1 y tomar una captura.
- Actualizar nuevamente hasta obtener APP2 y tomar una segunda captura.

TechPulse Proyecto académico de Software Avanzado.	Contenido asistido por IA Las publicaciones fueron generadas o asistidas con ChatGPT y se presentan exclusivamente con fines académicos. No constituyen información periodística verificada.	● Balanceador activo Instancia que respondió la verificación: APP1
--	--	---

TechPulse Proyecto académico de Software Avanzado.	Contenido asistido por IA Las publicaciones fueron generadas o asistidas con ChatGPT y se presentan exclusivamente con fines académicos. No constituyen información periodística verificada.	● Balanceador activo Instancia que respondió la verificación: APP2
--	--	---

El pie de página muestra la instancia que respondió la solicitud de verificación. Las capturas con APP1 y APP2 demuestran que existen al menos dos instancias funcionales detrás del balanceador.

EVIDENCIA 5

Prueba de balanceo mediante solicitudes consecutivas

Objetivo: Comprobar el algoritmo Round Robin mediante varias solicitudes sucesivas al balanceador.

Procedimiento para obtener la captura:

- Ejecutar en PowerShell el siguiente bloque y capturar toda la salida.

```
1..10 | ForEach-Object {  
    $instancia = Invoke-RestMethod http://localhost:8080/instancia  
    Write-Host "Solicitud $_ -> $instancia"  
}
```



```
PS C:\Users\Woe\Desktop\SA-Tarea2-NGINX-Portal-Noticias> 1..10 | ForEach-Object {  
>>     $instancia = Invoke-RestMethod http://localhost:8080/instancia  
>>     Write-Host "Solicitud $_ -> $instancia"  
>> }  
Solicitud 1 -> app1  
  
Solicitud 2 -> app2  
  
Solicitud 3 -> app1  
  
Solicitud 4 -> app2  
  
Solicitud 5 -> app1  
  
Solicitud 6 -> app2  
  
Solicitud 7 -> app1  
  
Solicitud 8 -> app2  
  
Solicitud 9 -> app1  
  
Solicitud 10 -> app2  
  
PS C:\Users\Woe\Desktop\SA-Tarea2-NGINX-Portal-Noticias>
```

La salida alterna las respuestas entre app1 y app2, evidenciando el uso de Round Robin. NGINX distribuye las solicitudes consecutivas entre los dos servidores definidos en el grupo upstream.

EVIDENCIA 6

Comportamiento al detener una instancia

Objetivo: Demostrar que el portal continúa disponible cuando una de las instancias deja de funcionar.

Procedimiento:

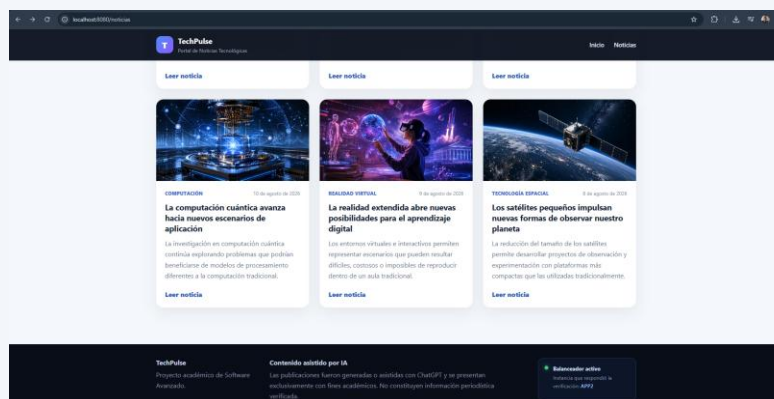
- Detener app1 y comprobar el estado de los contenedores.

```
docker stop techpulse-app1
docker compose ps
```

- Sin iniciar app1, recargar <http://localhost:8080> para comprobar que el portal sigue funcionando.
- Ejecutar la verificación de cinco solicitudes.

```
1..5 | ForEach-Object {
    Invoke-RestMethod http://localhost:8080/instancia
}
```

```
PS C:\Users\Woe\Desktop\SA-Tarea2-NGINX-Portal-Noticias> docker stop techpulse-app1
techpulse-app1
PS C:\Users\Woe\Desktop\SA-Tarea2-NGINX-Portal-Noticias> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
techpulse-app2      sa-tarea2-nginx-portal-noticias-app2  "/docker-entrypoint..."  app2       5 minutes ago  Up 5 minutes  80/tcp
techpulse-nginx     nginx:alpine         "/docker-entrypoint..."  nginx      5 minutes ago  Up 5 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
PS C:\Users\Woe\Desktop\SA-Tarea2-NGINX-Portal-Noticias> 1..5 | ForEach-Object {
>>     Invoke-RestMethod http://localhost:8080/instancia
>> }
app2
app2
app2
app2
app2
PS C:\Users\Woe\Desktop\SA-Tarea2-NGINX-Portal-Noticias>
PS C:\Users\Woe\Desktop\SA-Tarea2-NGINX-Portal-Noticias>
```



Al detener techpulse-app1, NGINX continúa enviando las solicitudes hacia app2 y el portal permanece disponible. Las verificaciones posteriores responden únicamente desde app2, demostrando tolerancia ante la caída de una instancia.

EVIDENCIA 7

Identificación de la herramienta de inteligencia artificial

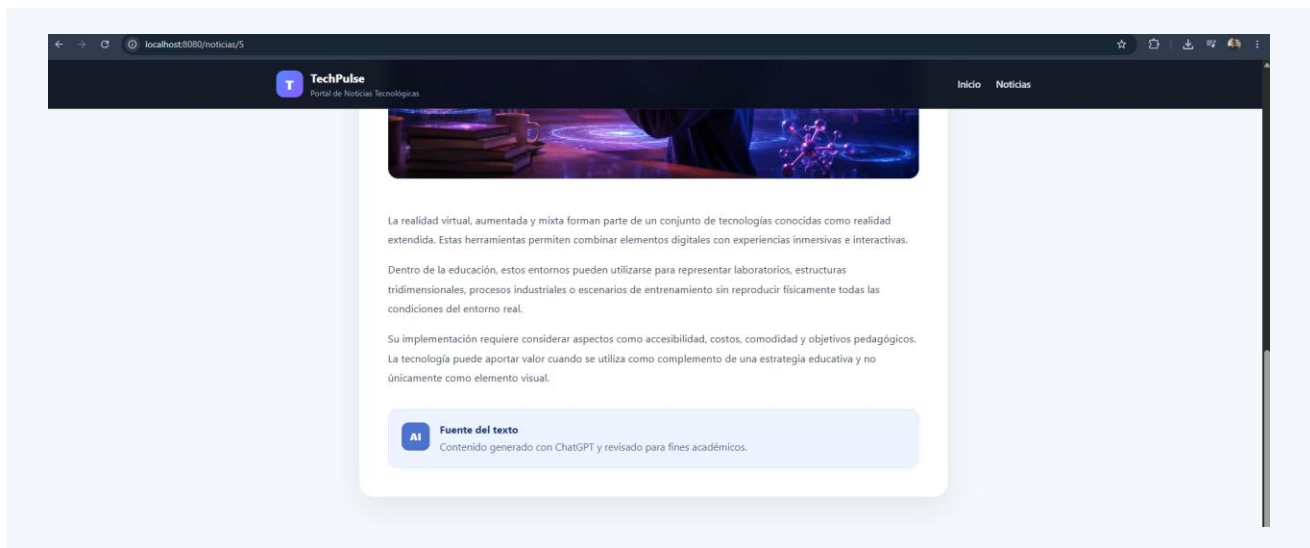
Objetivo: Demostrar que las noticias indican claramente el uso de ChatGPT y que el portal incluye la advertencia académica.

Procedimiento para obtener la captura:

- Abrir una noticia individual, por ejemplo:

```
http://localhost:8080/noticias/5
```

- Desplazarse hasta el bloque “Fuente del texto” y el pie de página.
- Tomar una captura donde sea legible la atribución a ChatGPT y la advertencia de contenido académico.

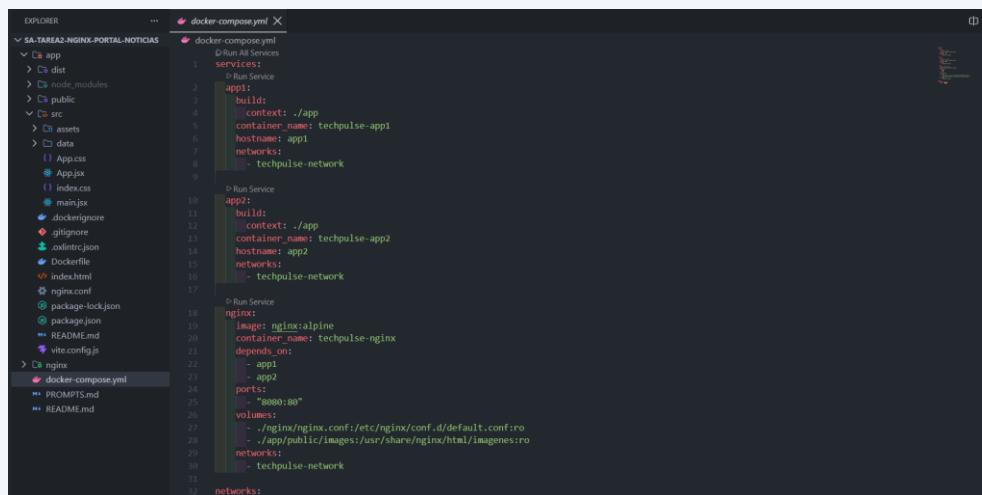


Cada noticia identifica la herramienta utilizada mediante la leyenda “Contenido generado con ChatGPT y revisado para fines académicos”. El pie de página también aclara que las publicaciones no constituyen información periodística verificada.

EVIDENCIA 8

Archivo docker-compose.yml

Objetivo: Presentar la configuración que define las dos instancias, NGINX y la red interna de Docker.



El archivo docker-compose.yml define los servicios app1, app2 y nginx, además de la red techpulse-network. Solo NGINX publica un puerto hacia el anfitrión, asegurando que el acceso a las instancias se realice mediante el proxy inverso.

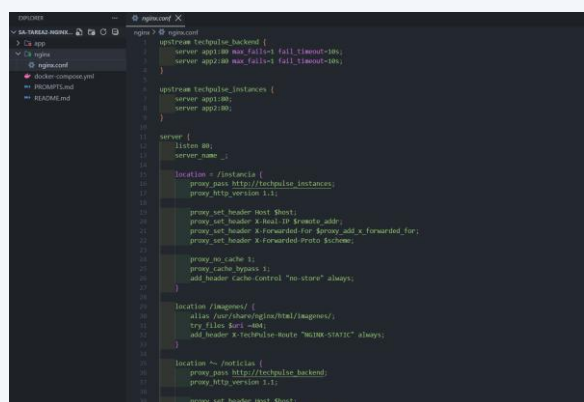
EVIDENCIA 9

Configuración de NGINX

Objetivo: Mostrar el grupo upstream, las reglas de enrutamiento y los encabezados reenviados por el proxy inverso.

Procedimiento para obtener la captura:

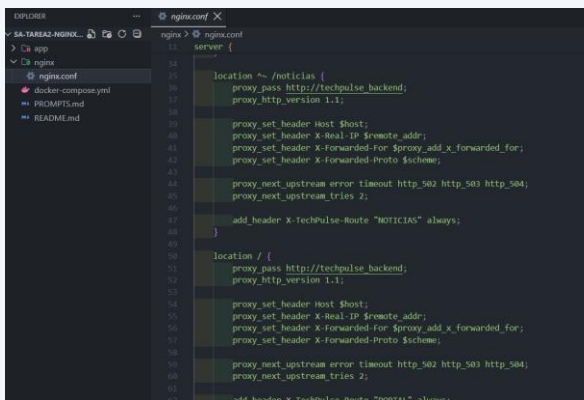
- Abrir nginx/nginx.conf en Visual Studio Code.
- Tomar una o dos capturas si es necesario para que sean legibles: upstream, location /noticias, location /imagenes/ y proxy_set_header.



```

1 upstream techpulse_backend {
2     server app1:80 max_fails=1 fail_timeout=10s;
3     server app2:80 max_fails=1 fail_timeout=10s;
4 }
5
6 upstream techpulse_instances {
7     server app1:80;
8     server app2:80;
9 }
10
11 server {
12     listen 80;
13     server_name _;
14
15     location ~ /instance {
16         proxy_pass http://techpulse_instances;
17         proxy_http_version 1.1;
18
19         proxy_set_header Host $host;
20         proxy_set_header X-Real-IP $remote_addr;
21         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
22         proxy_set_header X-Forwarded-Proto $scheme;
23
24         proxy_cache_bypass 1;
25         add_header Cache-Control "no-store" always;
26     }
27
28     location /imagenes/ {
29         alias /var/www/html/imagenes/;
30         try_files $uri -404;
31         add_header X-TechPulse-Route "IMAGES" always;
32     }
33
34     location ~ /noticias {
35         proxy_pass http://techpulse_backend;
36         proxy_http_version 1.1;
37
38         proxy_set_header Host $host;
39         proxy_set_header X-Real-IP $remote_addr;
40         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
41         proxy_set_header X-Forwarded-Proto $scheme;
42
43         proxy_next_upstream error timeout http_502 http_503 http_504;
44         proxy_next_upstream_tries 2;
45
46         add_header X-TechPulse-Route "NOTICIAS" always;
47     }
48
49     location / {
50         proxy_pass http://techpulse_backend;
51         proxy_http_version 1.1;
52
53         proxy_set_header Host $host;
54         proxy_set_header X-Real-IP $remote_addr;
55         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
56         proxy_set_header X-Forwarded-Proto $scheme;
57
58         proxy_next_upstream error timeout http_502 http_503 http_504;
59         proxy_next_upstream_tries 2;
60
61         add_header X-TechPulse-Route "PORTAL" always;
62     }
63 }

```



```

35     location ~ /noticias {
36         proxy_pass http://techpulse_backend;
37         proxy_http_version 1.1;
38
39         proxy_set_header Host $host;
40         proxy_set_header X-Real-IP $remote_addr;
41         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
42         proxy_set_header X-Forwarded-Proto $scheme;
43
44         proxy_next_upstream error timeout http_502 http_503 http_504;
45         proxy_next_upstream_tries 2;
46
47         add_header X-TechPulse-Route "NOTICIAS" always;
48     }
49
50     location / {
51         proxy_pass http://techpulse_backend;
52         proxy_http_version 1.1;
53
54         proxy_set_header Host $host;
55         proxy_set_header X-Real-IP $remote_addr;
56         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
57         proxy_set_header X-Forwarded-Proto $scheme;
58
59         proxy_next_upstream error timeout http_502 http_503 http_504;
60         proxy_next_upstream_tries 2;
61
62         add_header X-TechPulse-Route "PORTAL" always;
63     }

```

NGINX define un grupo upstream con app1 y app2 y utiliza Round Robin de forma predeterminada. La configuración incluye reglas para el portal, las noticias, la ruta estática /imagenes/ y la ruta /instancia, además de conservar encabezados relevantes de la solicitud original.

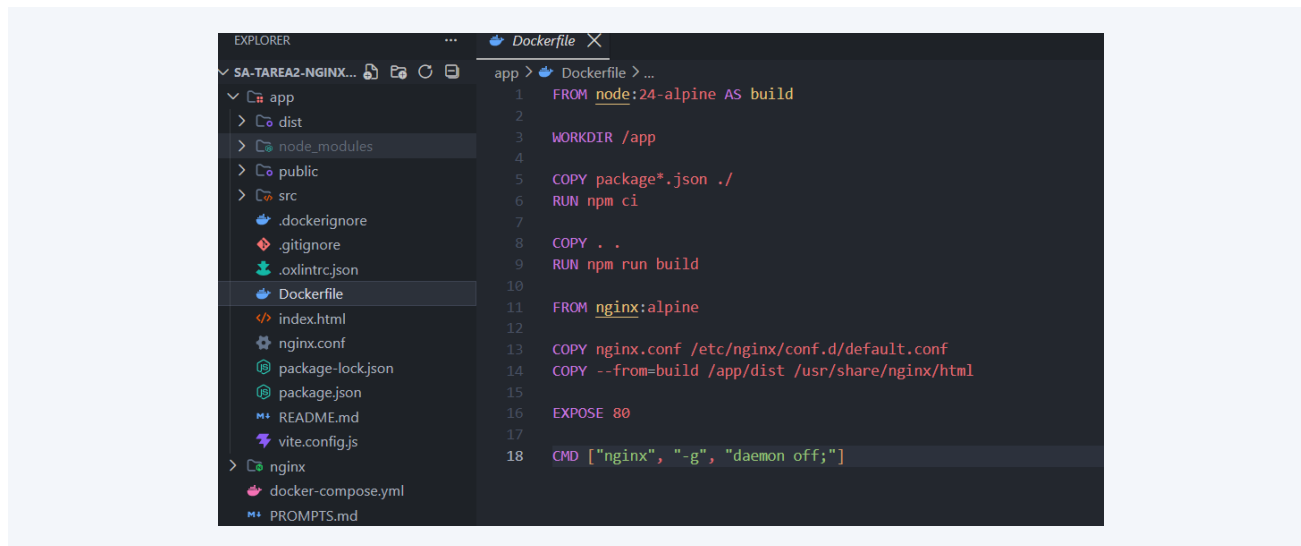
EVIDENCIA 10

Dockerfile de la aplicación

Objetivo: Demostrar que la aplicación posee un Dockerfile y que el proceso de construcción se realiza dentro de Docker.

Procedimiento para obtener la captura:

- Abrir app/Dockerfile en Visual Studio Code y tomar una captura completa.
- Debe observarse la etapa de Node.js para npm ci y npm run build, seguida de la etapa NGINX que sirve los archivos de producción.



El Dockerfile utiliza una construcción multietapa. La primera etapa compila la aplicación React con Node.js y la segunda utiliza NGINX para servir los archivos generados, evitando depender de una ejecución directa en el sistema anfitrión.

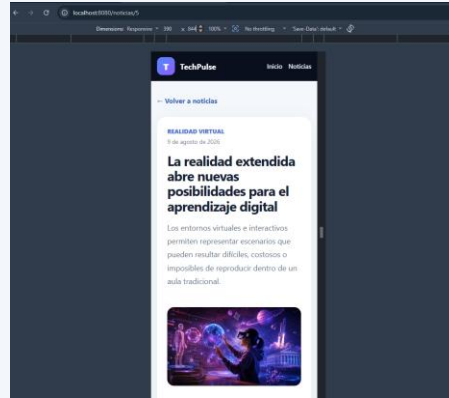
EVIDENCIA 11

Diseño adaptable a dispositivos móviles y tabletas

Objetivo: Aportar evidencia complementaria del comportamiento responsive requerido por la rúbrica.

Procedimiento para obtener la captura:

- Abrir las herramientas de desarrollo de Chrome y activar el modo de dispositivos.
- Capturar el portal en 390 x 844 (móvil).
- Capturar el listado de noticias en 768 x 1024 (tableta).



El diseño se adapta correctamente a diferentes tamaños de pantalla: en escritorio utiliza tres columnas, en tableta dos columnas y en móvil una sola columna. Los elementos de navegación, imágenes, contenido y pie de página permanecen legibles y ordenados.

Conclusiones

- La solución completa se ejecuta mediante contenedores y puede iniciarse con un único comando de Docker Compose.
- NGINX funciona como único punto de entrada, proxy inverso, sistema de enrutamiento y balanceador de carga.
- El algoritmo Round Robin distribuye las solicitudes entre dos instancias funcionales de la aplicación.
- El portal continúa disponible cuando una instancia se detiene, mientras la segunda permanece activa.
- Las noticias y sus imágenes identifican el uso académico de inteligencia artificial y el proyecto conserva los prompts utilizados.
- El diseño del portal es adaptable a computadoras, tabletas y dispositivos móviles.

Repositorio

Enlace del repositorio: <https://github.com/Noe94/SA-Tarea2-NGINX-Portal-Noticias.git>

Video demostrativo

Enlace del video (3 a 5 minutos):

<https://drive.google.com/file/d/19fuiFU3vZTTcyhweOirFkhND4UzJxrP/view?usp=sharing>